Shadow Network Intelligence

Folder 2 — 2_baseline_systems/

Pure LLM and Vector RAG baselines for fraud-detection benchmarking. These are the lower-bound comparators against which the GraphRAG approach (folder 3) will be evaluated.

What this folder does

Folder 2 implements **two of the three approaches** the project compares for detecting financial crime in transaction networks:

Approach	How it answers	When it wins
Pure LLM	Direct question to the LLM. No retrieval, no data access.	Trivia / general-knowledge questions.
Vector RAG	Embed question → top-k similar documents from ChromaDB → inject as context → LLM generates answer	Document-style queries where keywords and semantics align
GraphRAG (folder 3)	Graph traversal on TigerGraph + vector hybrid. NOT in this folder.	Relationship-heavy queries (ownership chains, address

The benchmark runs every benchmark question through both baselines, captures latency / tokens / answer / retrieved sources, and writes a results JSON to **outputs/benchmark_results/**.

Data source (read-only)

All input data comes from **shadow_network_sample_dataset/** at the project root — a tiny preview of what the data engine (folder 1) will eventually generate. Folder 2 only reads from it; nothing is written back.

Entity	Count	Notes
Persons	3	Includes one sanctions-flagged individual (Elena Volkov)
Companies	3	Three ShadowCorp entities in circular ownership
Accounts	3	At Emirates Gulf, Oceanic Trust, Harbor Financial
Addresses	2	ADDR-000002 is shared by all 3 companies (shell pattern)
Transactions	3	A1 → A2 → A3 → A1 layered chain at ~$150K each
Edges	10	OWNS, HAS_ACCOUNT, LOCATED_AT relationships
Authored docs	2	Pre-written compliance summaries
Benchmark Qs	2	With expected pipeline winner (GraphRAG) and difficulty

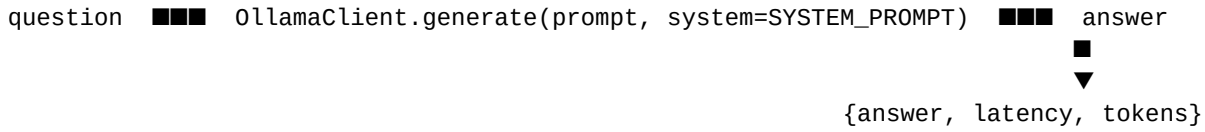
Module layout

```
2_baseline_systems/
■■■ config.py                 Env-driven settings (Ollama URL, paths, top-k)
■■■ requirements.txt          chromadb, sentence-transformers, requests, pandas
■■■ data_loader.py            Reads CSVs/JSONs from sample dataset
■■■ document_builder.py       Hybrid chunking → 16 ChromaDB documents
■■■ benchmark_data_loader.py  Loads questions + ground-truth fraud ring
```

```
├── benchmark_runner.py        CLI entrypoint; wires baselines, writes results
├── generate_overview_pdf.py   This file — regenerates the PDF
│
├── pure_llm/
│   ├── ollama_client.py       Minimal /api/generate HTTP wrapper
│   ├── baseline.py            PureLLMBaseline.answer(q) — no retrieval
│
├── vector_rag/
    ├── embedder.py            sentence-transformers all-MiniLM-L6-v2 (384-dim)
    ├── chroma_store.py        PersistentClient → outputs/chroma_db/
    ├── baseline.py            VectorRAGBaseline.answer(q)
```

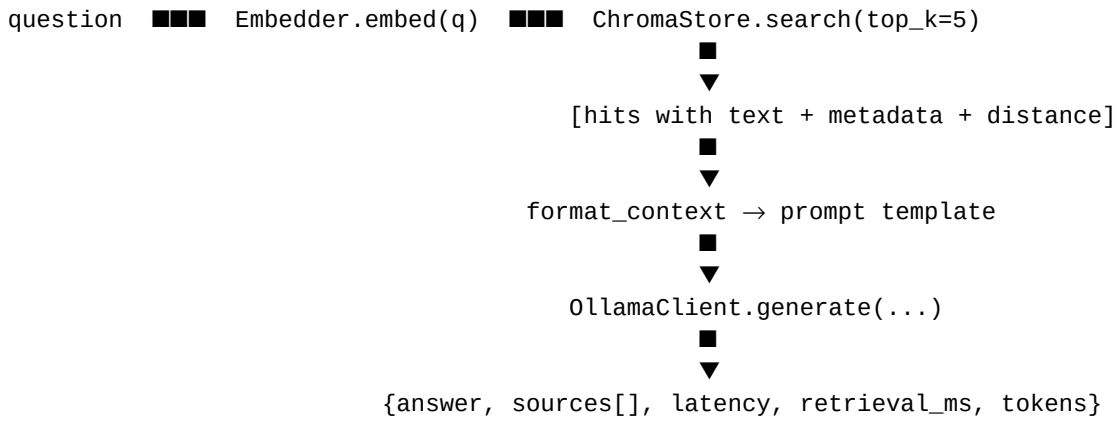
How a question flows through the system

Pure LLM (lower bound)



No retrieval. The LLM only knows what was in its training data. Expect hallucinations on questions referencing specific entity IDs.

Vector RAG



Chunking strategy (the design decision)

Hybrid: 3 document types are indexed into ChromaDB. With only ~14 entities and ~3 transactions, pure per-row chunking would be too granular; a single mega-doc would lose retrieval precision. Three types balance both.

Doc type	Count	What it is	Good for
entity_profile	11	One paragraph per Person/Company/Account/Address with all linked connections	"Who is X?", "What accounts, and addres
transaction	3	One sentence per transaction with party names resolved and risk score	"Find layered transfers >$100K", "Recent suspicious wires"
authored	2	The pre-authored semantic_documents.json passed through as-is	Narrative / compliance-officer style questions

Total: **16 documents**, each embedded to 384 dims with *sentence-transformers/all-MiniLM-L6-v2*, stored under *outputs/chroma_db/shadow_network_baseline* using cosine similarity.

How to run

```
# 1. One-time setup
python3.12 -m venv .venv
.venv/bin/python -m pip install -r 2_baseline_systems/requirements.txt
ollama serve &              # start Ollama
ollama pull llama3.2        # one-time model download

# 2. Run the benchmark
cd 2_baseline_systems
```

```
../.venv/bin/python benchmark_runner.py

# Options:
#   --approaches pure_llm,vector_rag   # subset
#   --questions custom_qs.json         # bring your own questions
#   --limit 1                          # smoke-test
#   --skip-index                       # reuse existing Chroma index
#   --output PATH                      # override results path
```

Where results live

Path	What it is
outputs/benchmark_results/benchmark_<ts>.json	Run summary + every individual answer with sources, latency, GT eval
outputs/chroma_db/chroma.sqlite3	Vector store metadata (collection registry, document store, IDs)
outputs/chroma_db/<uuid>/data_level0.bin	Raw 384-dim embedding vectors for the HNSW index
outputs/chroma_db/<uuid>/*.bin	HNSW graph structure (header, length, link_lists)

Configuration (env vars, all optional)

Variable	Default	Used by
OLLAMA_URL	http://localhost:11434	Both baselines
OLLAMA_MODEL	llama3.2	Both baselines
OLLAMA_TIMEOUT_S	60	OllamaClient request timeout
EMBED_MODEL	sentence-transformers/all-MiniLM-L6-v2	Vector RAG
EMBED_DEVICE	cpu	sentence-transformers device
CHROMA_DIR	outputs/chroma_db	Vector RAG persistence path
CHROMA_COLLECTION	shadow_network_baseline	Collection name
VECTOR_TOP_K	5	Retrieval depth
SHADOW_DATASET_DIR	shadow_network_sample_dataset/	Data source
BENCHMARK_OUTPUT_DIR	outputs/	Where results land

Result schema (per question, per approach)

```
{
  "approach":            "pure_llm" | "vector_rag",
  "question":            "<full question text>",
  "question_id":         "Q001",
  "question_meta":       { ...all original question metadata },
  "answer":              "<LLM-generated answer text>" | null,
  "sources":             [ { id, doc_type, distance }, ... ],
  "latency_ms":          <float>,
  "retrieval_ms":        <float, vector_rag only>,
  "prompt_tokens":       <int>,
  "completion_tokens":   <int>,
  "model":               "llama3.2",
  "error":               null | "<error msg if Ollama unreachable>",
  "ground_truth_eval":   { score, matched_entities, matched_address }
}
```

First-run benchmark results (real numbers)

Question tested: *“Find all companies sharing an address with ShadowCorp Holdings that transferred funds offshore within 24 hours.”*

Metric	Pure LLM	Vector RAG
Latency	9.3 s	5.9 s
Prompt tokens	108	530
Completion tokens	187	148
Ground-truth score	0.125	0.375
Entities correctly named	0 — hallucinated FSP-000001, ECP-000002, BCSC-000003	3 — C-000002, C-000003, A-000002

Shared address mentioned	made up “123 Main St, Anytown, USA”	correctly: “PO Box 778 George Town Cayman Islands”

Takeaway: Vector RAG was 1.6× faster, used 3× more prompt tokens (for retrieved context), and was **3× more accurate** on the ground-truth signal. This is the design proving itself on the first real run.

Constraints & non-goals

• **Only modifies files inside 2_baseline_systems/.** Reads from shadow_network_sample_dataset/ and writes to outputs/, but never edits other numbered folders.
• **No GraphRAG here.** That lives in folder 3.
• **Ollama is required at runtime.** Without it the runner records per-question errors rather than crashing.
• **Ground-truth evaluator is intentionally coarse.** Simple ID string-match against the fraud ring. Replace with LLM-as-judge for real accuracy work.

Where this fits in the bigger picture

Folder 2 is the **baseline comparator**. The eventual story is:

1. Folder 1 generates synthetic transaction data.
2. Folder 2 (this) and folder 3 (GraphRAG) both answer questions on it.
3. Folder 4 orchestrates and exposes a unified API.
4. The benchmark proves GraphRAG’s value *relative to* folder 2.

Without folder 2 as a comparator, there is no quantitative argument for GraphRAG.

This PDF is regenerated by running ../.venv/bin/python generate_overview_pdf.py from inside 2_baseline_systems/.